

Rapport simple du projet RT0806

Boutique sécurisée sur MQTT

Moussa William

10 juin 2026

1 Objectif du projet

Ce projet implémente une petite boutique en ligne sécurisée dont les échanges passent par un broker MQTT. Le serveur publie un catalogue de voitures, le client choisit un article, puis la commande est transmise de manière chiffrée.

L'objectif principal est de montrer comment combiner trois mécanismes de sécurité :

- une PKI locale pour authentifier le serveur ;
- RSA-OAEP pour transmettre une clé de session ;
- AES-256 pour chiffrer les messages applicatifs.

Le broker MQTT est lancé avec Docker Compose afin d'éviter d'installer Mosquitto directement sur la machine. Le serveur et le client restent des programmes Python exécutés localement.

2 Architecture générale

Le système contient trois parties principales :

- **Broker MQTT** : route les messages entre client et serveur. Il ne connaît pas le contenu des messages chiffrés.
- **Serveur** : génère la PKI locale, publie son certificat, reçoit les clés de session et traite les commandes.
- **Client** : vérifie le certificat serveur, crée une clé AES, reçoit le catalogue et envoie une commande chiffrée.

Le schéma suivant résume les échanges entre le client, le broker MQTT, le serveur et les modules de sécurité.

3 Rôle des fichiers

Fichier	Rôle
<code>docker-compose.yml</code>	Lance uniquement le broker MQTT Mosquitto dans un conteneur Docker.
<code>mosquitto.conf</code>	Configure Mosquitto sur le port 1883 avec accès anonyme pour le test local.
<code>pki.py</code>	Gère la CA locale, les certificats X.509, les clés RSA et la vérification du certificat serveur.
<code>core.py</code>	Contient les topics MQTT, la configuration réseau, l'enveloppement RSA-OAEP-SHA256 et le chiffrement AES-256-CBC.
<code>server.py</code>	Démarre le serveur, publie son certificat, déchiffre la clé AES du client, envoie le catalogue et reçoit les commandes.
<code>client.py</code>	Vérifie le certificat serveur, crée une clé AES, reçoit le catalogue, puis envoie une commande chiffrée.

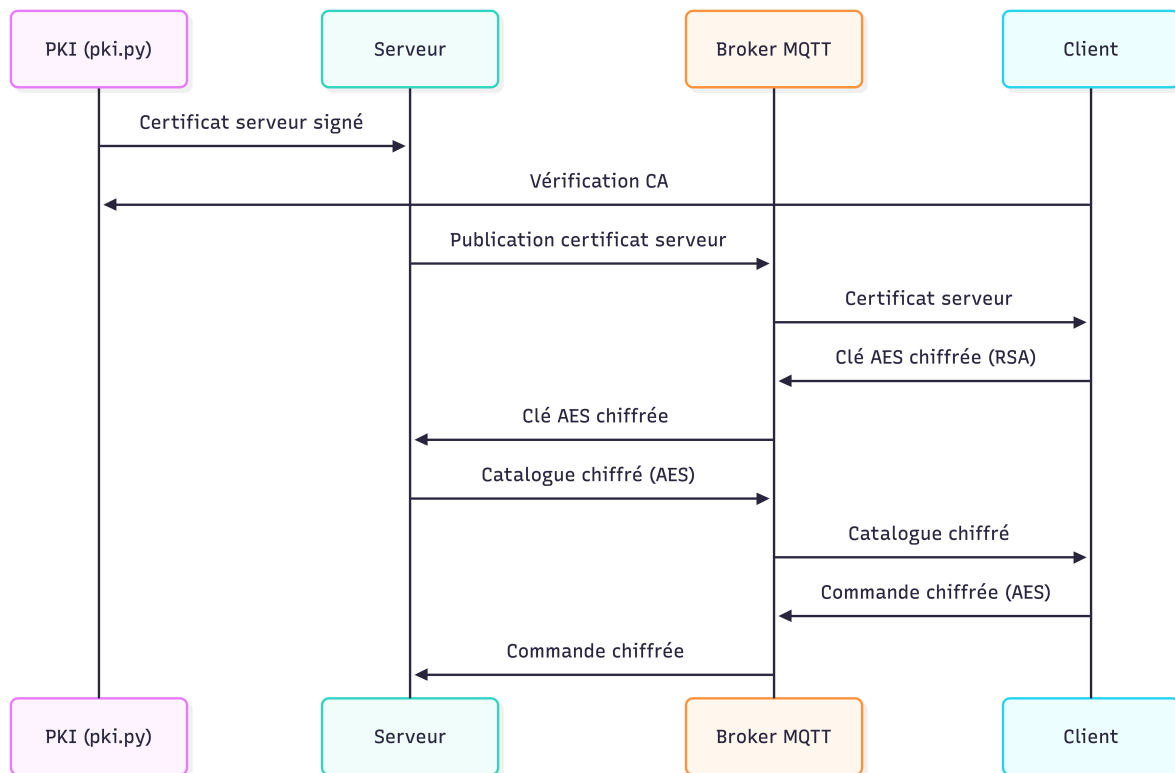


FIGURE 1 – Architecture générale et flux sécurisés du projet

4 Fonctionnement de la PKI

Au premier lancement du serveur, le module `pki.py` crée une petite autorité de certification locale :

- `ca_priv.pem` : clé privée de la CA ;
- `ca_cert.pem` : certificat public de la CA ;
- `server_priv.pem` : clé privée du serveur ;
- `server_cert.pem` : certificat serveur signé par la CA.

Le serveur publie ensuite `server_cert.pem` en base64 sur le topic MQTT `rt0806/srv/cert`. Le client reçoit ce certificat, charge `ca_cert.pem`, puis vérifie :

1. que le certificat serveur est émis par la CA locale ;
2. que sa signature est correcte ;
3. que le certificat est encore valide.

Si la vérification échoue, le client refuse le handshake. Sinon, il utilise la clé publique du certificat serveur pour transmettre une clé AES de session.

5 Séquence d'échange sécurisée

1. Le broker MQTT est lancé avec Docker Compose.
2. Le serveur se connecte au broker et publie son certificat X.509.
3. Le client reçoit le certificat et le vérifie avec la CA locale.
4. Le client génère une clé AES-256 aléatoire.
5. Le client chiffre cette clé avec RSA-OAEP-SHA256 et l'envoie au serveur.
6. Le serveur déchiffre la clé AES avec sa clé privée RSA.
7. Le serveur chiffre le catalogue avec AES-256-CBC et l'envoie au client.
8. Le client déchiffre le catalogue, choisit un article et envoie la commande chiffrée.
9. Le serveur déchiffre la commande et l'affiche dans ses journaux.

6 Topics MQTT utilisés

Topic	Utilisation
rt0806/srv/cert	Certificat serveur publié en message retained.
rt0806/auth/<cid>	Envoi de la clé AES enveloppée par RSA.
rt0806/auth/<cid>/ready	Confirmation de session par le serveur.
rt0806/products/<cid>	Catalogue chiffré envoyé au client.
rt0806/order/<cid>	Commande chiffrée envoyée au serveur.

7 Lancement du projet

Dans un premier terminal, lancer le broker MQTT :

```
docker compose up
```

Dans un deuxième terminal, lancer le serveur :

```
python3 server.py
```

Dans un troisième terminal, lancer le client :

```
python3 client.py
```

Il est aussi possible de choisir automatiquement un article :

```
python3 client.py --article-id 3
```

Pour arrêter le broker :

```
docker compose down
```

8 Limites et améliorations possibles

Le projet montre correctement les grandes étapes d'un échange sécurisé, mais il peut encore être amélioré :

- remplacer AES-CBC + SHA-1 par AES-GCM, qui fournit directement chiffrement et authentification ;
- protéger la clé privée de la CA avec un mot de passe ;
- séparer la CA du serveur dans un vrai environnement de production ;
- ajouter des tests automatisés pour la PKI, le chiffrement et les erreurs de certificat ;
- ajouter une authentification MQTT si le broker n'est pas limité à un usage local.

9 Conclusion

Ce projet met en place une architecture MQTT simple avec une couche de sécurité applicative. La PKI locale permet au client de vérifier l'identité du serveur, RSA-OAEP-SHA256 sert à établir une clé de session, puis AES-256-CBC chiffre le catalogue et la commande. Docker Compose simplifie uniquement le lancement du broker MQTT, tandis que le serveur et le client restent exécutés localement en Python.